Riphah International University Lahore, Pakistan



Riphah School of Computing & Innovation

PROJECT REPORT

AI-Powered Type Inference for Python

Project Team

Student Name	Student ID	Program	Contact Number	Email Address
ALEEZA NADEEM	48718	BSCS		
TEHREEM MAHMOOD	49140	BSCS		

Ms. Ayesha Kashif
(Senior Lecturer)

A Self-Evolving AI System for Adaptive Type

Inference in Python

Keywords

Static Code Analysis, Machine Learning, Type Inference, Software Engineering, Runtime Feedback, Random Forest

Abstract

Static type inference in dynamically typed languages remains a challenging problem due to limited compile-time information. This research proposes an adaptive machine learning–based framework for predicting variable types in Python programs using static code features. The approach begins with conservative static feature extraction using Abstract Syntax Tree (AST) analysis, followed by supervised learning through a Random Forest classifier. To address the inherent uncertainty of static analysis, an explicit unknown type is retained rather than forced predictions. The framework is further enhanced with project memory, runtime feedback, and periodic retraining, enabling continuous improvement without requiring expert annotation. Experimental results show that the model effectively predicts statically inferable types while correctly expressing uncertainty for dynamic cases. This work demonstrates that combining static analysis with adaptive learning provides a practical and extensible solution for assisting developers in dynamic programming environments.

Introduction

Dynamic programming languages such as Python offer flexibility and rapid development but lack explicit type information at compile time. This absence of static types complicates tasks such as code understanding, error detection, and automated refactoring. Traditional static analysis techniques struggle in such environments due to runtime-dependent behavior. Recent advances in machine learning have opened new opportunities to infer semantic properties of code from structural patterns. This research explores whether machine learning models can leverage static code features to predict variable types and how uncertainty can be handled systematically. The work builds on the idea that static inference does not need to be perfect to be useful, especially when combined with adaptive learning mechanisms.

Literature Review

Several approaches have been proposed for type inference and code understanding. Typilus (MIT CSAIL, 2020) uses Graph Neural Networks over program graphs but remains purely static and non-adaptive. Type4Py and TIGER generate and rank multiple candidate types but lack learning continuity across projects. QuAC focuses on object attribute inference, limiting its scope to partial program context. TypeGen adopts logic-inspired inference but does not personalize predictions. Overall, existing systems demonstrate strong performance in controlled settings but share limitations such as static behavior, lack of project-specific

adaptation, and limited handling of runtime information. These gaps motivate the development of an adaptive framework that integrates static learning with feedback-driven improvement.

Competitive Analysis

Attribute	Proposed Self-Evolving AI	Typilus	Type4Py / TIGER	QuAC	TypeGen
Inputs	Static features (AST), project memory, runtime feedback	Program graphs	Source code patterns	Program context	Program structure logic
Outputs	Predicted variable types	Type predictions	Multiple ranked candidate types	Object attribute inference	Type predictions
Primary Architecture	AST-based static analysis with adaptive learning	Graph-based neural modelling	ML-based ranking system	Contextual analysis framework	Logic-based inference system
Model Type	Random Forest classifier	Graph Neural Networks (GNN)	Machine learning based	Contextual analysis	Logic-inspired inference
Adaptability	**High** – periodic retraining and continuous evolution	None – purely static	Limited – no learning continuity	Static – limited scope	Static – fixed logic patterns
Personalisation	**High** – uses project memory	Low – no project-specific adaptation	Low – no project-specific adaptation	Limited – partial context only	None

Performance (Accuracy)	Good accuracy for statically inferable types; predicts **UNKNOWN** to avoid incorrect inference	~95% correctness when predicting, ~70% coverage	MRR ≈ 77% accuracy	~34% error-free non-trivial predictions	Improved exact-match accuracy over earlier systems
Error Handling	**Explicit UNKNOWN type** to safely handle uncertainty	No explicit uncertainty handling	Ranked outputs but no uncertainty modeling	Probabilistic but limited scope	No explicit uncertainty handling
Learning from Feedback	**Yes** – integrates developer corrections and runtime feedback	No	No	No	No
Deployment Practicality	**High** – lightweight, interpretable, low computational cost	Low – complex graphs and heavy models	Medium – deep learning overhead	Medium – limited scope	Medium – rigid logic system
Key Limitations	Cannot infer types dependent solely on runtime execution	Non-adaptive and strictly static	No continuity across codebases	Restricted to object attributes	Fails to capture project-specific patterns

Key Competitive Advantages of SETIS

Based on the sources, SETIS addresses three critical gaps left by its competitors:

- **Feedback Integration:** Unlike systems like **Typilus** or **Type4Py**, SETIS does not discard developer interactions. It stores corrected types in a **persistent memory file** to ensure the system "learns" from its mistakes rather than repeating them.
- **Contextual Evolution:** Traditional models are "fixed," meaning their accuracy can degrade as a codebase evolves with new patterns. SETIS uses **periodic retraining** to incorporate new data, allowing the model to grow alongside the project.

- **Interpretability vs. Complexity:** While competitors often rely on "black-box" deep learning or complex graph architectures, SETIS utilizes a **Random Forest** model. This provides a balance of **robustness and low computational cost**, making it easier to deploy in real-world environments where high-resource neural networks might be impractical.

Proposed Methodology

The proposed framework consists of four main phases. Phase 1 focuses on problem definition and feasibility analysis. Phase 2 performs static feature extraction from Python source code using AST analysis, collecting structural features and conservative type labels. Phase 3 applies supervised machine learning, specifically a Random Forest classifier, to learn relationships between code structure and variable types which further extends the system with analysis, interpretation, and adaptive learning mechanisms. The novelty of this work lies in combining conservative static inference with project memory, runtime feedback, and periodic retraining, enabling the model to evolve over time without manual supervision.

Data Collection, Preprocessing, and Model Implementation

Python source files were collected and parsed using AST-based static analysis. Features such as the number of assignments, control-flow constructs, and function calls were extracted. Each variable was represented as a separate data instance to improve label diversity. Preprocessing included removing invalid rows and encoding categorical labels numerically. A Random Forest classifier was trained using an 80:20 train-test split. Model performance was evaluated using accuracy, precision, recall, and F1-score. Visualizations such as accuracy comparison plots and confusion matrices were used to analyze behavior.

Results

The experimental results indicate that the model performs well for variables with clear static patterns, such as lists and dictionaries. Predictions for dynamically assigned variables often fall into the `unknown` category, reflecting appropriate uncertainty rather than incorrect inference. Training and testing accuracy values were closely aligned, suggesting good generalization. The confusion matrix revealed that most misclassifications occur between concrete types and the `unknown` class, which is expected in a dynamically typed language.

Discussion and Analysis

The results confirm that static structural features provide meaningful signals for type prediction. Cases where the model fails or predicts `unknown` are primarily due to runtime-dependent behavior, such as function return values and dynamic object creation. These findings have important implications for software engineering: static prediction systems should prioritize uncertainty awareness over forced correctness. The integration of runtime feedback allows the system to overcome static limitations progressively.

Limitations

The current framework cannot infer precise types for variables whose values depend entirely on runtime execution. Deep semantic relationships and inter-procedural data flow are not fully captured. Additionally, the model's performance depends on the quality of static features and available feedback. These limitations are acknowledged as inherent challenges in dynamic language analysis.

Future Work

Future work includes extending the framework to other programming languages, incorporating deep learning models, and improving feature-to-runtime mapping. Further exploration of clustering techniques for language-level analysis.

Conclusion

This research demonstrates that machine learning can effectively support static variable type prediction in dynamic languages when combined with conservative labeling and adaptive learning. Rather than eliminating uncertainty, the proposed framework manages it explicitly and reduces it over time through feedback. The results validate the feasibility of adaptive, learning-based type inference systems for practical applications.

References

Typilus:

Allamanis, M., Brockschmidt, M., & Khademi, M.,**"Learning to Represent Programs with Graphs"**, *International Conference on Learning Representations (ICLR)*, 2020.

🔗 Paper: https://arxiv.org/pdf/2004.10657

Type4Py:

Mir, A., Laredo, J., et al., **"Type4Py: Deep Similarity Learning-Based Type Inference for Python"**,
IEEE/ACM International Conference on Software Engineering (ICSE), 2021.

🔗 Paper: https://arxiv.org/abs/2101.04470

TIGER:

Wei, J., et al., **"TIGER: A Type Inference Engine with Generative Models"**, *arXiv preprint*, 2021.

🔗 Paper: https://arxiv.org/abs/2109.07474

QuAC:

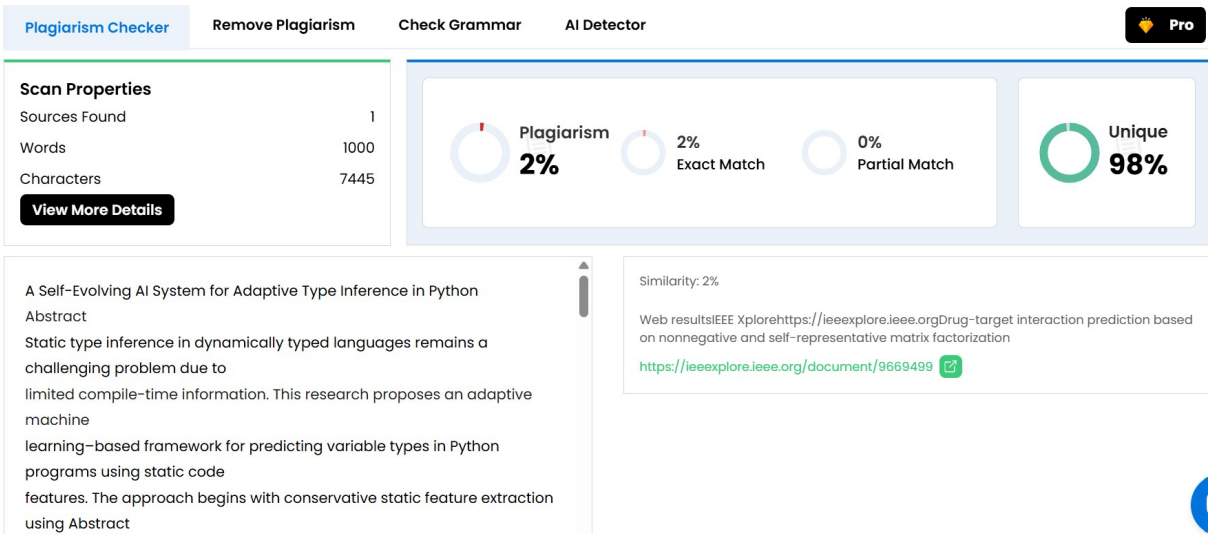
Zhang, J., et al., **“Probabilistic Type Inference for Python”**, *ACM Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2018. 🔗 Paper: https://dl.acm.org/doi/10.1145/3276483

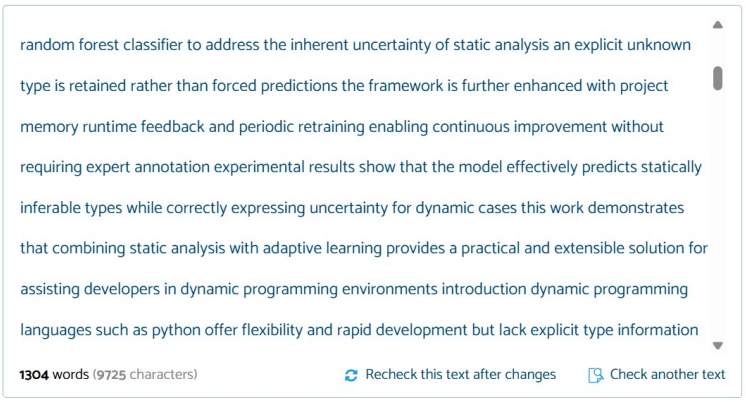
TypeGen:

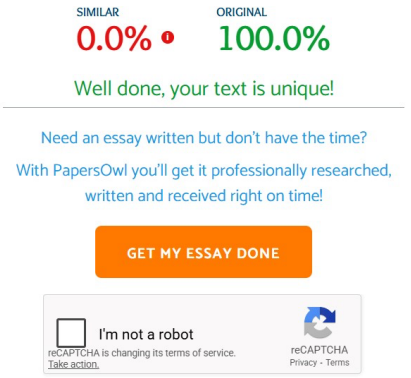
Xu, Z., et al., **“TypeGen: Type Inference for Python”**, *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2018.

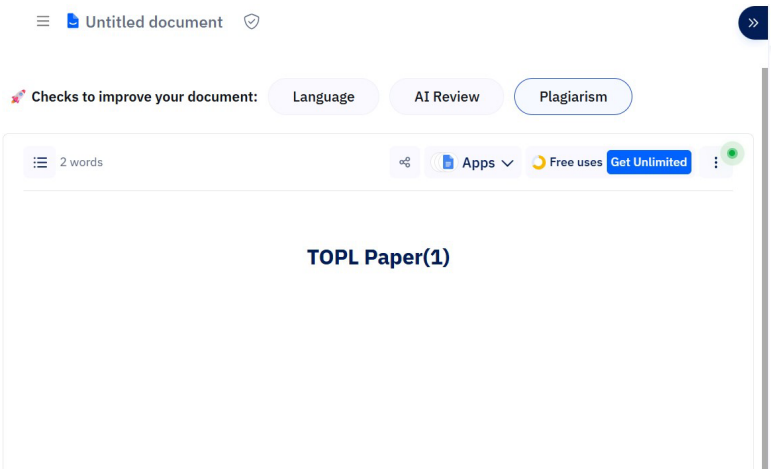
🔗 Paper: https://arxiv.org/abs/1801.01727

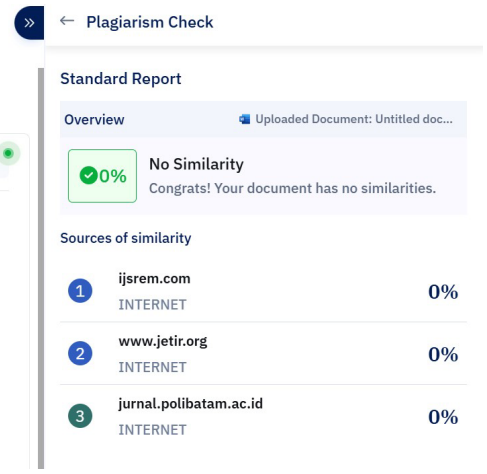
Plagiarism Report




random forest classifier to address the inherent uncertainty of static analysis an explicit unknown type is retained rather than forced predictions the framework is further enhanced with project memory runtime feedback and periodic retraining enabling continuous improvement without requiring expert annotation experimental results show that the model effectively predicts statically inferable types while correctly expressing uncertainty for dynamic cases this work demonstrates that combining static analysis with adaptive learning provides a practical and extensible solution for assisting developers in dynamic programming environments introduction dynamic programming languages such as python offer flexibility and rapid development but lack explicit type information
1304 words (9725 characters)
Recheck this text after changes
Check another text


SIMILAR
0.0%
ORIGINAL
100.0%
Well done, your text is unique!
Need an essay written but don't have the time?
With PapersOwl you'll get it professionally researched, written and received right on time!
GET MY ESSAY DONE
I'm not a robot
reCAPTCHA is changing its terms of service.
Take action.
reCAPTCHA
Privacy - Terms


Untitled document
Checks to improve your document:
Language
AI Review
Plagiarism
2 words
Apps
Free uses
Get Unlimited
TOPL Paper(1)


Plagiarism Check
Standard Report
Overview
Uploaded Document: Untitled doc...
0%
No Similarity
Congrats! Your document has no similarities.
Sources of similarity
1
ijsrem.com
INTERNET
0%
2
www.jetir.org
INTERNET
0%
3
jurnal.polibatam.ac.id
INTERNET
0%